



Manual del proyecto

Danivo CRM — CRM para talleres de reparación de celulares

Qué se construyó, cómo está organizado el código, qué fallas conocidas tiene y dónde corregirlas.
Generado el 27 de agosto de 2026 · para uso interno.

Índice

1. Panorama general
2. Mapa del código
3. Cómo publicar y trabajar en el proyecto
4. Base de datos (Supabase)
5. Funcionalidades — qué hace cada una, sus archivos y sus riesgos
6. Fallas conocidas y dónde corregirlas
7. Pendientes / roadmap
8. Cómo leer el código — patrones que se repiten

1 · Panorama general

Qué es. Danivo CRM es una aplicación web para gestionar un taller de reparación de celulares y electrónica: órdenes de servicio en un tablero, inventario de repuestos, ventas de accesorios, gastos, clientes (ficha automática), garantías con enlace público, y —en la capa "vendible como SaaS"— prueba gratis, cobro y un panel para el dueño del producto.

Stack

Framework	Next.js 15 (App Router) + React 19 + TypeScript
Estilos	Tailwind CSS (tokens en <code>tailwind.config.ts</code>)
Backend / datos	Supabase — Postgres + Auth + Storage. La seguridad la impone Row Level Security (RLS) en la base, no el código.
Hosting	Vercel (proyecto <code>kosmicksk12s-projects/orbitcrm</code>)
PWA	Instalable; el service worker offline quedó desactivado (ver \$5 y \$6).

Direcciones y cuentas

Producción	<code>https://orbitcrm-blond.vercel.app</code>
Repositorio	GitHub — <code>Kosmicksk12/orbitcrm</code> , rama <code>main</code>
Supabase ref	<code>pomelpmdslefqekvkyts</code>
Cuenta dueña / super-admin	<code>verdixinsta@gmail.com</code> (sembrada en <code>platform_admins</code>)

Cómo funciona el aislamiento entre talleres

Cada usuario pertenece a exactamente un **taller** (`shops`), con rol `admin` o `member` . Todas las tablas de datos tienen una columna `shop_id` y políticas RLS que solo dejan ver/editar filas del propio taller. Un usuario nunca ve datos de otro taller. El "super-admin" (dueño del SaaS) sí ve todo, pero solo a través de funciones especiales que chequean `is_platform_admin()` .

2 · Mapa del código

```
app/
  layout.tsx           raíz: fuentes, <head>, metadata, ToastProvider
  page.tsx             "/" → landing pública (o redirige a /dashboard si hay sesión)
  (auth)/             login, forgot-password, reset-password
  (dashboard)/
    layout.tsx         valida sesión, arma ShopProvider, Sidebar + Topbar + TrialBanner
    dashboard/page.tsx Panel (server component: consulta y calcula todo)
    orders/ sales/ expenses/ inventory/ clients/ settings/
    orders/[id]/warranty/ comprobante de garantía interno (para imprimir)
  admin/              /admin – panel de super-admin (layout propio, sin sidebar)
  garantia/[id]/      comprobante de garantía PÚBLICO (lo ve el cliente)
  terminos/ privacidad/ borradores legales con marcadores [TU NEGOCIO]

components/
  orders/ sales/ expenses/ inventory/ clients/ settings/ admin/   por dominio
  marketing/             LandingPage, LegalDoc
  layout/                Sidebar, Topbar, MobileNav, PageHeader, NotificationBell, TrialBanner
  ui/                    piezas base: Button, Modal, ConfirmDialog, Field, Primitives, Toaster,
                        Icons, ActionMenu, States, Logo
  pwa/                   InstallPrompt, ServiceWorkerRegister (hoy solo desregistra el SW viejo)
  shop/ShopContext.tsx   provee shopId, role, isAdmin, subscription, access, readOnly

lib/
  supabase/client.ts     cliente para componentes "use client" (navegador)
  supabase/server.ts     cliente para server components / middleware
  supabase/env.ts        lee las variables NEXT_PUBLIC_SUPABASE_*
  types.ts               interfaces TypeScript de todas las entidades
  utils.ts               formatCurrency, formatDate, buildWhatsAppLink, cn, ...
  subscription.ts        estado de la prueba/suscripción (writable / inTrial / expired)
  whatsapp.ts            arma y registra los mensajes de WhatsApp
  orderPhotos.ts         subir / borrar / firmar URLs de las fotos del equipo
  export.ts              exportar a Excel

hooks/useWriteGuard.ts   { readOnly, guard } – bloquea acciones si la prueba venció
middleware.ts            gating de auth en el edge de Vercel
supabase/*.sql           migraciones (se corren A MANO en el SQL Editor de Supabase)
scripts/gen-brand-icons.mjs regenera favicon + PNG de la marca desde design/*.svg
public/                  assets estáticos, manifest.webmanifest, sw.js (desactivado)
```

Rutas de la app

Ruta	Acceso	Qué es
/	público	Landing. Si hay sesión → /dashboard .
/login /forgot-password /reset-password	público	Autenticación.
/terminos /privacidad	público	Legales (borradores).
/garantia/[id]	público	Comprobante de garantía que se comparte con el cliente.
/dashboard /orders /sales /expenses /inventory /clients /settings	con sesión	La app del taller.
/orders/[id]/warranty	con sesión	Garantía interna, con fotos del equipo, para imprimir.
/admin	platform admin	Panel de super-admin. Cualquier otro → 404.

3 · Cómo publicar y trabajar en el proyecto

Publicación (autorizada automática, ver `CLAUDE.md`)

El flujo para cada cambio es **verificar** → **commit** → **push a `main`** → **deploy**. Antes de cada push:

1. `npx tsc --noEmit` — debe salir limpio.
2. `npx next lint` — debe salir limpio.
3. `npx next build` — debe compilar sin errores. **Nunca correr `next build` con `next dev` activo** — corrompe la carpeta `.next`.
4. Si algo falla, NO publicar. Arreglar primero.
5. Solo si los tres pasan: `git add`, `git commit`, `git push origin main`, `npx vercel --prod`.
6. Después del deploy, verificar rápido en `https://orbitcrm-blond.vercel.app` que no haya errores.

Correr en local

```
npm install
npm run dev           # http://localhost:3000 (desarrollo, con hot reload)
npm run build && npm start # simula producción (más fiel)
```

Aviso: `next dev` a veces no muestra componentes nuevos aunque el código esté bien (artefacto del hot reload). Si algo nuevo "no aparece" pero `tsc` y `next build` pasan, probá contra `npm run build && npm start` antes de asumir que hay un bug.

Migraciones de base de datos

El código nunca ejecuta SQL de esquema. El flujo es: escribir el archivo en `supabase/migration-*.sql`, abrir el **SQL Editor de Supabase**, pegar el contenido y darle "Run" a mano. Los archivos `.sql` quedan en el repo solo como respaldo y referencia.

Variables de entorno (`.env.local` en local, panel de Vercel en prod)

Variable	Para qué
<code>NEXT_PUBLIC_SUPABASE_URL</code>	URL del proyecto Supabase
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code> / <code>_PUBLISHABLE_KEY</code>	Llave pública de Supabase (segura de exponer — la seguridad la da RLS)
<code>NEXT_PUBLIC_SITE_URL</code>	URL base para armar el enlace público de garantía
<code>NEXT_PUBLIC_BILLING_ENABLED</code>	Prende el cobro con Stripe. Hoy está sin definir (= apagado).

4 · Base de datos (Supabase)

Tablas y para qué sirven

Tabla	Qué guarda
<code>profiles</code>	Una fila por usuario (nombre, email, <code>shop_name</code> — este último quedó a medio migrar, ver §6).
<code>shops</code>	El taller. Nombre y —desde la migración de suscripción— <code>trial_ends_at</code> , <code>subscription_status</code> , <code>stripe_customer_id</code> , <code>stripe_subscription_id</code> .
<code>shop_members</code>	Qué usuario pertenece a qué taller y con qué rol (<code>admin</code> / <code>member</code>).
<code>shop_invites</code>	Invitaciones por email pendientes. Al registrarse alguien con ese email, un trigger lo mete al taller.
<code>service_orders</code>	El corazón: una fila por reparación. <code>status</code> , montos en centavos, garantía en días, <code>deleted_at</code> (papelera).
<code>inventory_products</code>	Repuestos y accesorios con stock y precios.
<code>sales</code> / <code>sale_items</code>	Ventas de accesorios (no reparaciones). Descuentan stock vía RPC.
<code>expenses</code>	Gastos del negocio. Tiene <code>deleted_at</code> (papelera).
<code>requested_products</code>	Encargos de clientes que no hay en stock.
<code>whatsapp_messages</code>	Registro (solo-agregar) de cada WhatsApp que se abrió desde la app.
<code>order_photos</code>	Metadatos de las fotos del equipo. Los archivos viven en Storage, bucket privado <code>order-photos</code> .
<code>platform_admins</code>	Los dueños del SaaS. Habilita <code>/admin</code> .

Migraciones que hay que tener corridas

El esquema base (`schema.sql`) + estas migraciones, en orden aproximado. Las `nuevas` son de estas sesiones y hay que confirmarlas en Supabase:

Archivo	Qué agrega
migration-v1-to-v2 / v2-to-v3-teams	Evolución vieja: multiusuario y roles por taller.
migration-add-inventory / add-sales / sales-custom-items / add-sale-costs	Inventario y ventas.
migration-add-expenses	Gastos.
migration-public-warranty	RPC <code>get_public_warranty</code> .
migration-requested-products	Encargos.
migration-add-soft-delete nueva	<code>deleted_at</code> en <code>service_orders</code> y <code>expenses</code> ; redefine <code>get_public_warranty</code> para excluir borrados.
migration-add-whatsapp-log nueva	Tabla <code>whatsapp_messages</code> + RLS.
migration-add-order-photos nueva	Bucket <code>order-photos</code> + policies de <code>storage.objects</code> + tabla <code>order_photos</code> .
migration-add-subscription nueva	Columnas de prueba/suscripción en <code>shops</code> ; talleres existentes quedan en <code>active</code> .
migration-add-platform-admin nueva	Tabla <code>platform_admins</code> + funciones <code>is_platform_admin</code> , <code>admin_list_shops</code> , <code>admin_set_shop_subscription</code> . Siembra a <code>verdixinsta@gmail.com</code> .
migration-unify-shop-name pendiente	Todavía NO existe en <code>main</code> . Está en la rama <code>claude/angry-kapitsa-c491bd</code> . Ver §6.

Cómo funciona RLS (el modelo de seguridad)

- `user_shop_ids()` — función que devuelve el/los `shop_id` del usuario actual. Casi toda política dice `shop_id in (select public.user_shop_ids())`.
- `is_shop_admin(shop_id)` — true si el usuario es admin de ese taller. Las políticas de `DELETE` suelen exigirlo.
- `is_platform_admin()` — true si el usuario está en `platform_admins`. Lo chequean las funciones del panel `/admin`.
- Los `SELECT` casi siempre son abiertos para miembros del taller; `INSERT` / `UPDATE` también; `DELETE` solo admins.

Funciones RPC (lógica que corre en la base)

Función	Qué hace
<code>create_sale / update_sale / delete_sale</code>	Registrar/editar/eliminar una venta ajustando el stock en la misma transacción. <code>delete_sale</code> repone el stock.
<code>get_public_warranty(order_id)</code>	Datos mínimos de una orden para el comprobante público, sin exponer la tabla entera. Excluye órdenes en la papelera.
<code>is_platform_admin()</code>	Chequeo de super-admin.
<code>admin_list_shops()</code>	Una fila por taller con métricas (miembros, órdenes, ventas, última orden, email del dueño). Solo platform admins.
<code>admin_set_shop_subscription(shop_id, status, trial_ends_at?)</code>	Cambiar a mano el estado de un taller (activar, extender prueba, marcar vencida). Útil para activar un taller que pagó por transferencia mientras no haya Stripe.

5 · Funcionalidades — qué hace cada una, sus archivos y sus riesgos

5.1 Papelera (soft delete) de órdenes y gastos

Qué hace. "Eliminar" no borra: marca la fila con `deleted_at` y la saca de las vistas normales. Desde la papelera se puede restaurar o "borrar para siempre" (solo admins).

Archivos. `components/orders/OrdersTrashModal.tsx`, `components/expenses/ExpensesTrashModal.tsx`; el filtro `.is("deleted_at", null)` está en todas las lecturas (Panel, Clientes, buscador, cierre de mes, notificaciones, garantía).

Riesgos. Si en el futuro se agrega una lectura de órdenes o gastos *sin* el filtro `.is("deleted_at", null)`, van a aparecer filas de la papelera donde no deberían. Ventas **no** tiene papelera a propósito (ahí "eliminar" repone stock y mezclar eso con restaurar es delicado).

5.2 Historial de WhatsApp

Qué hace. Cada vez que se abre WhatsApp desde la app (lista de órdenes, tarjeta kanban, comprobante de garantía) se registra el mensaje prellenado en `whatsapp_messages`. Se ve en el modal de la orden y en la ficha de cliente (agrupado por teléfono).

Archivos. `components/orders/WhatsAppLink.tsx` (registra al hacer clic), `components/orders/WhatsAppHistory.tsx` (lista), `lib/whatsapp.ts`. Tabla `whatsapp_messages`.

Riesgos. El registro es "fire and forget": si falla no muestra error. Y solo registra que *se abrió* WhatsApp, no que el cliente lo haya recibido o leído — el texto de la UI lo aclara, pero conviene no venderlo como "mensajes enviados".

5.3 Fotos del equipo (Supabase Storage)

Qué hace. Evidencia del estado del equipo al recibirlo. Se pueden adjuntar al crear la orden (quedan en memoria y se suben después del *insert*, con el `order_id` nuevo) o al editarla. Se ven en el modal y en la garantía interna; **nunca** en la garantía pública. Máx. 12 fotos por orden; se comprimen en el navegador antes de subir.

Archivos. `components/orders/OrderPhotos.tsx`, `lib/orderPhotos.ts`. Bucket privado `order-photos`, ruta `{shop_id}/{order_id}/{uuid}`. Tabla `order_photos`. Las miniaturas usan URLs firmadas (el bucket es privado).

Riesgos. (1) Las policies de `storage.objects` filtran por el **primer segmento de la ruta** (`shop_id`): si algún código sube con una ruta distinta, la subida se rechaza — es la parte más delicada de seguridad. (2) Borrar una orden "para siempre" limpia los archivos de Storage (`deleteAllOrderPhotoFiles` en el trash modal); si esa llamada falla quedan archivos huérfanos (costo menor, no seguridad). (3) Las URLs firmadas vencen (1 h) — si alguien deja el modal abierto mucho rato, las miniaturas dejan de cargar hasta recargar.

5.4 Landing pública + páginas legales

Qué hace. `/` muestra una landing de marketing para visitantes sin sesión. `/terminos` y `/privacidad` son borradores con marcadores `[TU NEGOCIO]`, `[FECHA]`, etc.

Archivos. `components/marketing/LandingPage.tsx`, `components/marketing/LegalDoc.tsx`, `app/page.tsx`, `app/terminos/page.tsx`, `app/privacidad/page.tsx`. El `middleware.ts` deja `/terminos` y `/privacidad` como rutas públicas.

Riesgos. Los legales **tienen que completarse con datos reales** antes de cobrarle a nadie — hoy son plantillas. Los precios de la landing son presentacionales (todos los botones van a `/login`).

5.5 Prueba de 15 días → modo solo-lectura

Qué hace. Un taller nuevo arranca con 15 días de acceso total (`subscription_status = 'trialing'` , `trial_ends_at = now() + 15 días`). Al vencer, si no hay suscripción activa, pasa a **solo lectura**: pueden entrar y ver todo, pero no crear/editar/borrar. Los talleres existentes quedaron en `active` por la migración.

Archivos. `lib/subscription.ts` (la lógica: `resolveAccess()` → `writable / inTrial / expired`), `components/shop/ShopContext.tsx` (expone `readOnly`), `hooks/useWriteGuard.ts` , `components/layout/TrialBanner.tsx` , `components/settings/SubscriptionSettings.tsx` . El bloqueo está aplicado en órdenes, ventas, gastos, inventario, productos solicitados, fotos y gestión de equipo.

Riesgos importantes. El bloqueo solo-lectura es **solo del lado del cliente** (deshabilita botones y corta los handlers). No hay RLS que lo respalde todavía: un usuario técnico con la prueba vencida podría, en teoría, escribir llamando a la API de Supabase directo. Es un hueco conocido y aceptable *por ahora* (no hay clientes externos con prueba vencida). Se cierra cuando se conecte Stripe (ver §6 y §7). El `ShopProvider` hace *fail open*: si la columna no existe o la consulta falla, el taller se trata como `active` — para no bloquear a nadie por un error.

5.6 Panel de super-admin (/admin)

Qué hace. Para el dueño del SaaS: ver todos los talleres con su estado de prueba/suscripción, contadores (miembros, órdenes, ventas), correo del dueño y última actividad; y cambiar el estado de un taller a mano (activar / extender prueba 15 días / marcar vencida).

Archivos. `app/admin/layout.tsx` , `app/admin/page.tsx` (chequea `platform_admins` ; si no, `notFound()` → 404), `components/admin/AdminDashboard.tsx` . Usa los RPC `admin_list_shops` y `admin_set_shop_subscription` . El link "Panel de administración" aparece en el menú de usuario solo si sos platform admin (el `(dashboard)/layout.tsx` lo pasa a `Topbar`).

Riesgos. Para agregar otro super-admin hay que insertar una fila en `platform_admins` a mano en Supabase (no hay UI). El panel confía en que los RPC hacen el chequeo de autorización — si alguien crea un RPC nuevo para el panel, tiene que empezar con `if not public.is_platform_admin() then raise exception ...` .

5.7 Guía de primeros pasos (onboarding)

Qué hace. Un taller nuevo ve una tarjeta "Primeros pasos" en el Panel con checklist (nombre del taller, primera orden, primer producto, invitar equipo). Desaparece sola cuando los 3 pasos obligatorios están hechos, o con "Ocultar" (se guarda en `localStorage` del dispositivo).

Archivos. `components/dashboard/OnboardingChecklist.tsx` ; `app/(dashboard)/dashboard/page.tsx` calcula el progreso de datos que ya consulta.

Riesgos. "Ocultar" es por-dispositivo (no por-cuenta). El check "nombre del taller" mira `profiles.shop_name` o `shops.name` — se simplifica cuando se unifique el nombre (§6).

5.8 Rediseño de marca

Qué hace. El logo pasó de la elipse tipo "ojo" a una **"D" geométrica** blanca sobre el cuadrado azul `#2647E0` . Wordmark unificado: Space Grotesk 700 con `letter-spacing -0.03em` en toda la app.

Archivos. `components/ui/Logo.tsx` (un solo `<svg>`), los `<span>` del wordmark en `Sidebar/Topbar/login/landing/legales/admin`, `design/icon-source.svg` + `design/icon-maskable-source.svg` , `scripts/gen-brand-icons.mjs` (regenera `favicon.ico` y los PNG de `public/icons/`), `public/manifest.webmanifest` .

Riesgos. (1) Los PNG de `public/icons/` van con caché de un año (`immutable`): para cambiarlos de nuevo hay que **renombrarlos** (por eso llevan sufijo `-r2`) y actualizar el manifest + `app/layout.tsx` . La constante `REV` en el script controla ese sufijo. (2) Dispositivos con la PWA ya instalada: el ícono de la pantalla de inicio no cambia hasta **reiniciar** la PWA (en iOS es obligatorio). (3) Las pantallas de arranque `public/splash/*` todavía tienen la marca vieja (§6).

5.9 Endurecimiento del middleware (fix del error 504)

Qué hace. El `middleware.ts` llamaba a `supabase.auth.getUser()` en cada request sin límite de tiempo propio; una demora de Supabase se convertía en `504 MIDDLEWARE_INVOCATION_TIMEOUT` de Vercel. Ahora: (1) `Promise.race` con timeout de 2,5 s — si no responde, el request pasa sin redirigir (los layouts del área privada igual chequean la sesión al renderizar); (2) se saltea Supabase del todo para rutas que no lo necesitan (`/` , legales, `/garantia/*` , assets).

Riesgos. Si el timeout salta, ese request no refresca la cookie de sesión (se refresca en el siguiente) y un usuario logueado que caiga en `/login` justo en ese momento ve el login en vez de rebotar al dashboard. Es raro y aceptable.

5.10 Service worker desactivado

Qué pasó. Tras un cambio de versión del SW + reinstalación de la PWA, el service worker quedaba "envenenado" y servía `/offline.html` en bucle, dejando la app sin cargar aunque el servidor respondiera. Se desactivó: `public/sw.js` ahora se autodesregistra, borra todas las cachés y recarga las pestañas; `components/pwa/ServiceWorkerRegister.tsx` solo desregistra SW viejos.

Consecuencia. Se perdió el modo offline. Si se quiere de vuelta, hay que reintroducir un SW con **estrategia robusta**: nombres de assets versionados, `skipWaiting` automático, y que un 404 de un chunk no deje una página rota cacheada.

6 · Fallas conocidas y dónde corregirlas

#	Síntoma	Causa	Dónde / cómo corregir	Gravedad
1	El comprobante de garantía que ve el cliente dice "Danivo CRM" aunque el taller se puso nombre. En <code>/admin</code> aparece "Sin nombre".	Ajustes escribe <code>profiles.shop_name</code> , pero la garantía pública (<code>get_public_warranty</code>) y <code>/admin (admin_list_shops)</code> leen <code>shops.name</code> . Están desincronizados.	El fix ya está hecho en el commit <code>eead1da</code> de la rama <code>claudie/angry-kapitsa-c491bd</code> (SettingsClient lee/escribe <code>shops.name</code> solo si <code>isAdmin</code> ; WarrantyClient lee <code>shops.name</code> ; dashboard usa <code>shops.name</code> ; migración <code>migration-unify-shop-name.sql</code> rellena la columna). Esa rama es vieja: cherry-pick solo eead1da sobre main actual , resolver los conflictos chicos en esos 3 archivos, correr la migración, publicar.	media
2	Al abrir la PWA instalada, la pantalla de arranque (splash) tiene la marca vieja.	Los 7 PNG de <code>public/splash/</code> son de antes del rediseño.	Regenerarlos con la "D" nueva (fondo <code>#F1F3F9</code> , mark centrado) a las dimensiones exactas por dispositivo. Extender <code>scripts/gen-brand-icons.mjs</code> o hacer <code>scripts/gen-splash.mjs</code> con <code>sharp</code> . Las referencias están en <code>app/layout.tsx</code> (<code><link rel="apple-touch-startup-image"></code>).	baja (cosmético)
3	Un usuario con la prueba vencida podría escribir datos igual, saltándose la UI.	El bloqueo solo-lectura es solo del lado cliente. No hay RLS que lo respalde.	Al conectar Stripe: agregar una función <code>shop_is_writable(shop_id)</code> (<code>= subscription_status = 'active'</code> OR prueba vigente) y meterla en las políticas de <code>INSERT / UPDATE / DELETE</code> de <code>service_orders</code> , <code>inventory_products</code> , <code>sales</code> , <code>sale_items</code> , <code>expenses</code> , <code>requested_products</code> , <code>order_photos</code> , <code>whatsapp_messages</code> , <code>shop_invites</code> , <code>shop_members</code> ; y en los RPC <code>create_sale / update_sale / delete_sale</code> .	media (hasta que haya clientes reales)
4	El botón "Suscríbete" solo muestra "los pagos se activan pronto".	Stripe no está conectado. El andamiaje está (flag <code>NEXT_PUBLIC_BILLING_ENABLED</code> , columnas <code>stripe_*</code> en <code>shops</code> , el botón).	Crear cuenta Stripe → poner las llaves en variables de entorno de Vercel (NUNCA en el repo) → endpoint <code>/api/billing/checkout</code> → webhook que actualiza <code>subscription_status</code> → RLS del punto 3. Mientras tanto, activar talleres a mano desde <code>/admin</code> .	pendiente planificado
5	Las páginas <code>/terminos</code> y <code>/privacidad</code> tienen marcadores como <code>[TU NEGOCIO]</code> .	Son borradores a propósito.	Reemplazar los marcadores por datos legales reales (razón social, dirección, correo, jurisdicción, proveedor de pagos) en <code>app/terminos/page.tsx</code> y <code>app/privacidad/page.tsx</code> antes de cobrar.	bloqueante para cobrar
6	<code>npm audit</code> reporta 2 vulnerabilidades.	PostCSS interno de Next.js. Riesgo real bajo para esta app.	Se resolverían saltando a Next 16 — se decidió NO hacerlo por ahora (salto mayor sin necesidad). Revisar cuando haya otra razón para actualizar.	baja

7	Cambiás un ícono/splash y en producción sigue el viejo.	<code>next.config.mjs</code> cachea <code>/icons/:path*</code> y <code>/splash/:path*</code> como <code>immutable</code> (un año).	Renombrar el archivo (sufijo de versión) y actualizar sus referencias. El script de íconos ya lo hace con la constante <code>REV</code> .	baja (operativa)
8	En <code>next dev</code> un componente nuevo "no aparece" aunque el código esté bien.	Artefacto del hot reload / compilación incremental de Next. Pasó varias veces en este proyecto.	No es un bug del código. Verificar contra <code>npm run build && npm start</code> . Si ahí funciona, está bien.	no es bug
9	Los números de orden tienen saltos (ej. FP-000012 no existe).	Se crearon órdenes de prueba durante el desarrollo y se borraron; la secuencia <code>service_orders_seq</code> no retrocede.	Cosmético, no se toca. Postgres nunca reusa números de secuencia.	cosmético
10	Hay ramas viejas en el repo (<code>claude/adoring-mcclintock-...</code> , <code>upgrade/nextjs-15</code>).	Trabajo previo que quedó sin limpiar.	No mergear <code>claude/adoring-mcclintock-*</code> (su diff borra el panel admin, onboarding, legales). Borrarla. <code>upgrade/nextjs-15</code> ya está en <code>main</code> — borrarla. <code>claude/angry-kapitsa-*</code> conservar hasta aplicar el fix del punto 1, después borrarla.	higiene

7 · Pendientes / roadmap

- **Conectar Stripe** (checkout + webhook + endurecer el solo-lectura con RLS). Es lo único grande que falta de la fase "SaaS vendible".
- **Unificar** `shop_name` (punto 1 de §6) — chico, ya casi hecho en una rama.
- **Splash screens** con la marca nueva (punto 2 de §6) — cosmético.
- **Completar los legales** con datos reales antes de cobrar.
- Ideas mencionadas sin compromiso: recordatorios automáticos de garantía/orden lista, firma digital del cliente, reportes comparativos mes a mes, multi-moneda (hoy fijo en COP), reintroducir soporte offline con un SW robusto.

8 · Cómo leer el código — patrones que se repiten

Server Component vs Client Component

Por defecto los archivos de `app/` son **server components**: corren en el servidor, pueden usar `await` y consultar Supabase con `lib/supabase/server.ts`. Un archivo que empieza con `"use client"` corre en el navegador: puede usar `useState`, `useEffect`, eventos, y consulta con `lib/supabase/client.ts`. El Panel (`dashboard/page.tsx`) es server; casi todo lo demás interactivo es un "PageClient".

El patrón "PageClient"

```
"use client";
export function OrdersBoard() {
  const supabase = createClient();
  const { shopId, isAdmin } = useShop();
  const { readOnly } = useWriteGuard();
  const { toast } = useToast();
  const [orders, setOrders] = useState([]);

  async function load() {
    // consulta y llena el estado
    const { data } = await supabase.from("service_orders")
      .select("*, is('deleted_at', null).order('created_at', { ascending: false }));
    setOrders(data ?? []);
  }
  useEffect(() => { load(); }, []);

  async function handleSubmit(values) {
    // los handlers escriben y vuelven a load()
    if (readOnly) return; // guard de solo-lectura
    await supabase.from("service_orders").insert({ ...values, shop_id: shopId });
    load();
  }
}
```

Cosas que se repiten

- **Plata en centavos.** Las columnas terminan en `_cents` y son enteros. Para mostrar: `formatCurrency(cents)` de `lib/utils.ts`.
- **Soft delete.** Toda lectura de órdenes/gastos lleva `.is("deleted_at", null)`.
- **Contexto del taller.** `const { shopId, role, isAdmin, readOnly, access, subscription } = useShop()` — disponible en cualquier componente dentro de `(dashboard)`.
- **Bloqueo de escritura.** `const { readOnly } = useWriteGuard()`; los handlers empiezan con `if (readOnly) return;` y los botones llevan `disabled={readOnly}`.
- **Modales.** `<Modal>` para formularios; `<ConfirmDialog>` para confirmar acciones destructivas (nunca `window.confirm`).
- **Avisos.** `const { toast } = useToast(); toast({ title, description, variant })` — variantes: `"default"`, `"success"`, `"danger"`.
- **Íconos.** Componentes SVG en `components/ui/Icons.tsx`: `<IconWrench width={16} height={16} />`.
- **Tailwind.** Las clases usan tokens: `bg-surface`, `text-ink`, `border-line`, `text-accent`, y su par `dark:`. Definidos en `tailwind.config.ts`.
- **Tipos.** `lib/types.ts` tiene la interfaz de cada entidad (`ServiceOrder`, `Sale`, `Expense`, `InventoryProduct`, `WhatsAppMessage`, `OrderPhoto`, ...).

Para agregar una funcionalidad nueva típica

1. Si necesita tabla nueva: escribir `supabase/migration-*.sql` con la tabla, sus índices y las políticas RLS (copiar el patrón de `migration-requested-products.sql`: `SELECT/INSERT/UPDATE` para miembros, `DELETE` para admins). Correrla a mano en Supabase.
2. Agregar la interfaz TypeScript en `lib/types.ts`.
3. Crear el/los componentes en `components/<dominio>` siguiendo el patrón PageClient.
4. Si escribe datos, cortar con `useWriteGuard()` y deshabilitar los botones con `readOnly`.
5. `npx tsc --noEmit + npx next lint + npx next build → commit → push → npx vercel --prod`.

Este manual describe el estado del proyecto al 27 de agosto de 2026. El código evoluciona: ante la duda, la fuente de verdad es el repo y el historial de `git log`.